

問題1-2: PRBS から QAM シンボルへの変換(QAMマッピング)

問題

問1で作った **周期 32767 ビットの PRBS** を k 回繰り返し、グレイ符号化された **4QAM / 16QAM / 64QAM / 256QAM** のシンボル列へ変換する。シンボル数はいずれも $2^{15} - 1 = 32767$ とする。繰り返し数 k は QAM の次数(1シンボルあたりのビット数)に等しく、

方式	1シンボルのビット数 k	1軸のレベル数 $L = \sqrt{M}$
4QAM	2	2
16QAM	4	4
64QAM	6	8
256QAM	8	16

平均強度(平均シンボル電力)が 1 になるよう規格化し、コンステレーション(複素平面の散布図)を確認する。このビット列→シンボル列の変換を **QAMマッピング** と呼ぶ。

記号について — 問題文では繰り返し数を M と書いているが、QAMの多値数(4,16,64,256)と紛らわしいので、本解説では **多値数を M 、1シンボルのビット数(=繰り返し数)を $k = \log_2 M$** と書く。

実行結果(出力)

PRBS 周期 : 32767 ビット

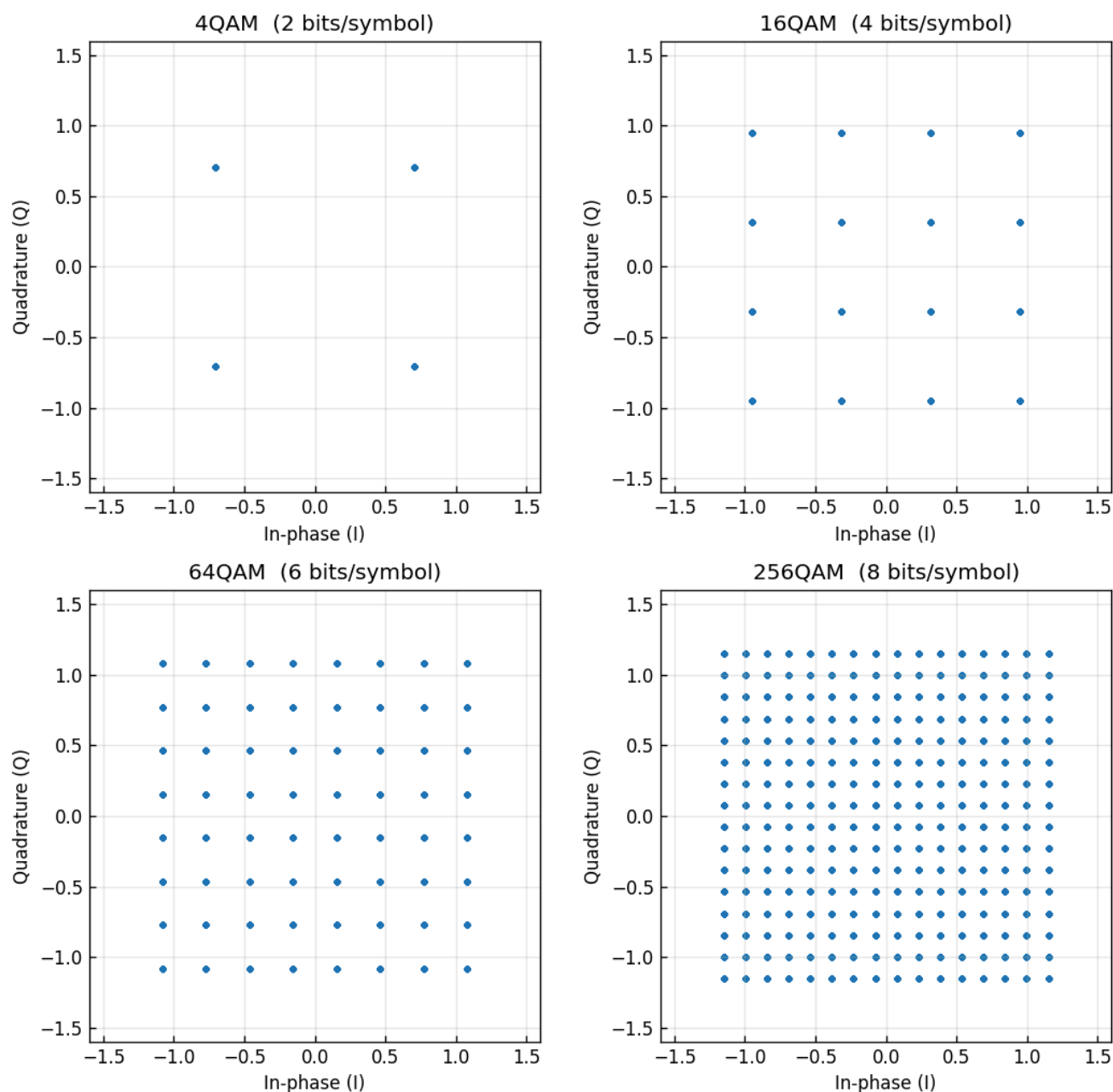
4QAM: ビット/シンボル=2, シンボル数=32767, 平均電力=1.0000, I軸レベル数=2

16QAM: ビット/シンボル=4, シンボル数=32767, 平均電力=1.0000, I軸レベル数=4

64QAM: ビット/シンボル=6, シンボル数=32767, 平均電力=1.0000, I軸レベル数=8

256QAM: ビット/シンボル=8, シンボル数=32767, 平均電力=0.9999, I軸レベル数=16

QAM constellations from PRBS15 (average power = 1)



図の読み方 — 各点が1つのシンボル。QAMの次数が上がるほど格子が細くなり、隣り合う点の間隔が狭くなる(= 雑音に弱くなる)。すべて平均電力1に規格化してあるので、外周の点の半径はどの方式でもおおよそ同じ範囲に収まっている。

1. QAM とは — I/Q 平面の格子点

QAM (Quadrature Amplitude Modulation, 直交振幅変調) は、搬送波の **同相成分 (In-phase, I)** と **直交成分 (Quadrature, Q)** の2つを独立に振幅変調する方式である。複素ベースバンドで表すと、1シンボルは

$$s = a_I + j a_Q$$

という1つの複素数になる。 a_I, a_Q がそれぞれ離散的な振幅レベルをとる。**方形 (square) QAM** では、I軸・Q軸が同じ $L = \sqrt{M}$ 値の **PAM (Pulse Amplitude Modulation)** になっていて、振幅レベルは奇数の等間隔値

$$a \in \{-(L-1), \dots, -1, +1, \dots, (L-1)\}$$

をとる。例えば 16QAM なら $L = 4$ で、各軸が $\{-3, -1, +1, +3\}$ の4レベル。I と Q を組み合わせて $4 \times 4 = 16$ 通りの格子点ができる。

つまり $M\text{-QAM} = (\sqrt{M}\text{値PAM}) \times (\sqrt{M}\text{値PAM})$ と分解できる。これが後の処理(マッピング・判定)を「I軸とQ軸を別々に扱えばよい」という簡単な形にしてくれる。

2. グレイ符号化 — なぜ必要か

各格子点には $k = \log_2 M$ ビットを割り当てる。割り当て方には自由度があるが、**隣り合う点どうしが必ず1ビットだけ違う** ように割り当てるのが **グレイ符号 (Gray code)** である。

なぜ重要か。受信側で雑音に乗ると、判定ミスは **隣の点に間違える** ことが圧倒的に多い。グレイ符号なら「隣に間違える=ビット誤りは1個だけ」で済むので、**シンボル誤り1個あたりのビット誤りが最小**になる。これがそのまま BER(ビット誤り率)を下げる。

本実装では、 k ビットを前半 $k/2$ ビット(I軸)と後半 $k/2$ ビット(Q軸)に分け、各軸を独立にグレイ符号化している。1軸(L 値PAM)のグレイ符号化は次の手順:

ビット(グレイ符号) --グレイ→2進--> レベル番号 idx ($0 \dots L-1$) --> 振幅 $a = 2 \cdot \text{idx} - (L-1)$

グレイ→2進 変換は $b = g \wedge (g \gg 1) \wedge (g \gg 2) \wedge \dots$ (共通ライブラリ `comm._gray_decode`)。デマッピング(問1-5)では逆に 2進→グレイ 変換 $g = b \wedge (b \gg 1)$ を使う。

例:16QAM の I軸(4値, 2ビット)のグレイ割り当て

ビット(グレイ)	2進index	振幅 a_I
00	0	-3
01	1	-1
11	2	+1
10	3	+3

振幅が1段変わるときにビットが1個だけ変化していることが確認できる。

3. 平均電力1への規格化

レベル $\{\pm 1, \pm 3, \dots, \pm(L-1)\}$ をそのまま使うと、平均シンボル電力は方式ごとにバラバラになる。1軸(L 値PAM、各レベル等確率)の平均電力は

$$\mathbb{E}[a^2] = \frac{1}{L} \sum_{i=0}^{L-1} (2i - (L-1))^2 = \frac{L^2 - 1}{3} = \frac{M - 1}{3}.$$

I軸とQ軸の2軸ぶんで、規格化前の平均シンボル電力は

$$\mathbb{E}[|s|^2] = 2 \cdot \frac{M-1}{3} = \frac{2(M-1)}{3}.$$

そこで全シンボルを $\sqrt{2(M-1)/3}$ で割れば、平均電力がちょうど 1 になる (共通ライブラリ `comm.qam_norm`)。こうしておくで、異なるQAM方式どうしを **同じSNRの土俵**で比較できる(問1-3以降で雑音を加えるときに効いてくる)。

出力の「平均電力=1.0000」は実測値。256QAM だけ 0.9999 なのは、PRBS が完全な乱数ではなく 有限長の系列なので、各レベルの出現回数がわずかに偏るため。系列を長くすれば 1 に近づく。

4. 「PRBS を k 回繰り返す」の意味

シンボル数を $2^{15} - 1 = 32767$ に固定したいので、必要なビット数は $k \times 32767$ になる。PRBS は1周期 32767 ビットしか無いので、これを k 回タイル状に繰り返して必要本数をそろえる (コードの `np.tile(prbs, k)`)。

```
4QAM    : 32767 ビット × 2回 = 65534 ビット → 32767 シンボル
256QAM   : 32767 ビット × 8回 = 262136 ビット → 32767 シンボル
```

こうすると、どの方式でも **シンボル数が同じ 32767** になり、後段の比較が公平になる。

5. コードのポイント

- `comm.generate_prbs(15)` — 問1の LFSR を共通ライブラリ化したもの。
- `np.tile(prbs, k)` — PRBS を k 回繰り返してビット列を作る。
- `comm.bits_to_symbols(bits, M)` — QAMマッピング本体。bits を k ビットずつ区切り、前半をI軸・後半をQ軸のグレイ符号として複素シンボルに変換し、`qam_norm(M)` で規格化する。
- `ax.scatter(sym.real, sym.imag)` — コンステレーション(I-Q平面散布図)を描く。

実行

```
python solution.py
```

標準出力は冒頭の「実行結果」、生成図は `constellation.png` を参照。

6. この問題のつながり

ここで作った QAM シンボル列が、以降の問題の出発点になる。

- **問1-3:** このシンボル列に AWGN(白色ガウス雑音)を加えて受信を模擬する。
- **問1-4:** 雑音に乗ったシンボルを最近傍判定(decision)する。
- **問1-5:** 判定結果をビットに戻す(QAMデマッピング)。
- **問1-6:** 送信ビットと比べてビット誤り率(BER)を求める。